Atividade da Semana — Reescrita de Algoritmos em Rust

Disciplina: Estruturas de Dados e Análise de Algoritmos
Aula: 04 – Implementação de Algoritmos em Rust
Professor: Alexandre de Oliveira
Entrega: Postar no ULife em data agendada

Contexto

Na aula anterior iniciamos o estudo de análise assintótica de algoritmos e discutimos como estimar a complexidade de tempo utilizando a notação Big-O.

Para praticar esses conceitos, trabalhamos com algoritmos escritos em Python, analisando:

- Quantas operações são executadas;
- Como o tempo cresce conforme o tamanho da entrada;
- Qual é a classe de complexidade assintótica do algoritmo.

Nesta atividade, o objetivo é reforçar esse raciocínio e ampliar o repertório técnico, **reimplementando os algoritmos em Rust**, uma linguagem moderna amplamente utilizada em sistemas de alto desempenho.

Objetivos da Atividade

1. **Consolidar** o entendimento da análise de complexidade de algoritmos
2. **Praticar** a leitura e interpretação de código
3. **Introduzir** o uso da linguagem Rust para implementação de algoritmos

---

Tarefa

Você deverá reescrever em Rust os algoritmos analisados na aula anterior e realizar as seguintes etapas para **cada um** deles:

1 — Analisar o algoritmo original

Para cada algoritmo em Python:

- Leia o código e entenda seu comportamento
- Identifique a estrutura principal (loops, recursão, condicionais)
- Determine a complexidade Big-O no pior caso
- Explique brevemente:
  - Quantos loops existem e como estão organizados
  - Como o tamanho da entrada `n` influencia o tempo de execução
  - Qual a classe de complexidade (ex: `O(n)`, `O(n²)`, `O(log n)`, etc.)

2 — Reescrever o algoritmo em Rust

Reescreva cada algoritmo mantendo a mesma lógica do original.

Requisitos:

- O código deve compilar sem erros (`cargo build`)
- O algoritmo deve produzir o mesmo resultado que a versão Python
- Utilize tipagem explícita (ex: `i32`, `usize`, `Vec<i32>`)
- Comente o código para facilitar a leitura

3 — Executar testes

Execute o algoritmo e apresente evidências de funcionamento.

Sugestões:

- Testar com diferentes tamanhos de entrada
- Mostrar a saída no terminal com `println!`
- Escrever ao menos um `#[test]` por função

Algoritmos para Reescrever

A seguir estão os 10 algoritmos em Python. Para cada um há também dicas específicas de Rust para orientar a implementação.

Exercício 1 — Verificar Primeiro

```python
def verificar_primeiro(lista):
    if len(lista) == 0:
        return None
    return lista[0]
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Retornos opcionais com `Option<T>`

Em Python, `None` é um valor como qualquer outro. Em Rust, a ausência de valor é representada pelo tipo `Option<T>`, que pode ser `Some(valor)` ou `None`. O compilador obriga você a tratar os dois casos.

```rust
// Assinatura sugerida
pub fn verificar_primeiro(lista: &[i32]) -> Option<i32>
```

- `&[i32]` é uma **referência a uma fatia (slice)** — você não precisa tomar posse do vetor, apenas emprestá-lo para leitura.
- O método `.first()` já retorna `Option<&i32>`. Use `.copied()` para obter `Option<i32>`.
- Alternativa manual: verifique `lista.is_empty()` e retorne `None`, ou acesse `lista[0]` e retorne `Some(lista[0])`.

Exercício 2 — Somar Lista

```
def somar_lista(lista):
    total = 0
    for elemento in lista:
        total += elemento
    return total
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Empréstimo (`&`) e desestruturação no `for`

```
// Assinatura sugerida
pub fn somar_lista(lista: &[i32]) -> i32
```

- Passe a lista por **referência imutável** (`&[i32]`): a função apenas lê os dados, não precisa de posse.
- Para iterar e obter o valor diretamente (não uma referência), desestruture com `&`:

```
for &elemento in lista {
    total += elemento;
}
```

- Variáveis mutáveis exigem `mut` explícito: `let mut total = 0;`
- O compilador Rust garante que `total` não foi modificado de fora do escopo sem permissão.

Exercício 3 — Busca Binária

```
def busca_binaria(lista, alvo):
    esquerda, direita = 0, len(lista) - 1
    while esquerda <= direita:
        meio = (esquerda + direita) // 2
        if lista[meio] == alvo:
            return meio
        elif lista[meio] < alvo:
```

```
            esquerda = meio + 1
        else:
            direita = meio - 1
    return -1
```

Complexidade esperada: `O( __ )`

> **Atenção:** a lista deve estar ordenada em ordem crescente para que a busca binária funcione corretamente.

▶ 💡 Dicas de Rust

Aritmética de índices com tipos com sinal (`isize`)

```rust
// Assinatura sugerida
pub fn busca_binaria(lista: &[i32], alvo: i32) -> Option<usize>
```

- Índices de slices em Rust são do tipo `usize` (sem sinal), mas `direita` pode chegar a `-1` na lógica da busca binária. Use `isize` para as variáveis de controle e converta com `as usize` ao acessar o slice.
- Em vez de retornar `-1` (impossível em `usize`), use `Option<usize>`: `Some(idx)` quando encontrar, `None` quando não encontrar.
- Múltiplos retornos (`return`) dentro de um `while` são perfeitamente válidos em Rust.

```rust
let mut esquerda: isize = 0;
let mut direita: isize = lista.len() as isize - 1;
// ...
let idx = meio as usize; // converte para acessar o slice
```

Exercício 4 — Pares com Soma

```python
def pares_com_soma(lista, alvo):
    for i in range(len(lista)):
        for j in range(i + 1, len(lista)):
            if lista[i] + lista[j] == alvo:
                print(lista[i], lista[j])
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Ranges e loops aninhados

```rust
// Assinatura sugerida
pub fn pares_com_soma(lista: &[i32], alvo: i32)
```

- Ranges em Rust: `0..n` (exclui `n`) e `(i+1)..n`. São idiomáticos e não há custo extra.
- Acesso por índice: `lista[i]` — Rust verifica os limites em tempo de execução no modo debug.
- Para formatar a saída: `println!("{} + {} = {}", lista[i], lista[j], alvo);`
- **Desafio opcional:** retornar os pares em um `Vec<(i32, i32)>` em vez de apenas imprimir — isso facilita testes.

```rust
for i in 0..n {
    for j in (i + 1)..n {
        // ...
    }
}
```

Exercício 5 — Imprimir Pares e Pares

```python
def imprimir_pares_e_soma(lista):
    # Bloco 1: imprime cada elemento
    for i in range(len(lista)):
        print(lista[i])

    # Bloco 2: imprime todos os pares
    for i in range(len(lista)):
        for j in range(len(lista)):
            print(lista[i], lista[j])
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Regra da soma e blocos sequenciais

```rust
// Assinatura sugerida
pub fn imprimir_pares_e_pares(lista: &[i32])
```

- Este exercício tem dois blocos de código sequenciais. Em Rust, assim como na análise Big-O, eles simplesmente se sucedem — não há nenhuma sintaxe especial.
- Itere com `for &x in lista` para obter o valor diretamente.
- O Bloco 2 usa dois loops aninhados sobre a **mesma** slice. Note que em Python o `range(len(lista))` começa do zero nos dois loops — aqui todos os pares `(i, j)` são gerados, inclusive `(i, i)`.

```rust
// Bloco 1 — O(n)
for &x in lista {
    println!("{}", x);
}

// Bloco 2 — O(n²)
for &x in lista {
    for &y in lista {
        println!("({}, {})", x, y);
    }
}
```

Exercício 6 — Potências de Dois

```python
def potencias_de_dois(n):
    i = 1
    while i < n:
        print(i)
        i *= 2
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Tipos inteiros sem sinal e `while` com mutação

```rust
// Assinatura sugerida
pub fn potencias_de_dois(n: u64)
```

- Use `u64` (inteiro sem sinal de 64 bits): potências de dois crescem rapidamente e `u64` suporta valores até ~1,8 × 10^{19}.
- `i *= 2` é equivalente a `i = i * 2`; ambos são válidos em Rust.
- Declare a variável de controle como mutável: `let mut i: u64 = 1;`
- Em modo debug, Rust detecta overflow de inteiros em tempo de execução (panic). Em modo release, o comportamento é wrapping. Para este exercício, `u64` é suficientemente grande.

```rust
let mut i: u64 = 1;
while i < n {
    println!("{}", i);
    i *= 2;
}
```

Exercício 7 — Fibonacci Recursivo

```python
def fibonacci_recursivo(n):
    if n <= 1:
        return n
    return fibonacci_recursivo(n - 1) + fibonacci_recursivo(n - 2)
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Recursão e casos base

```rust
// Assinatura sugerida
pub fn fibonacci_recursivo(n: u64) -> u64
```

- Recursão em Rust funciona exatamente como em Python. O compilador não otimiza tail-call por padrão, mas para este algoritmo não é necessário.
- Use `u64`: os valores de Fibonacci crescem exponencialmente e rapidamente ultrapassam o limite de `i32`.
- O caso base `if n <= 1 { return n; }` é a âncora que impede recursão infinita.
- **Atenção ao desempenho:** este algoritmo recalcula os mesmos subproblemas repetidamente. Para `n > 40`, o tempo de execução se torna perceptível. **Não teste com valores altos.**
- **Desafio opcional:** implemente uma versão iterativa com `O(n)` e compare os resultados.

```rust
pub fn fibonacci_recursivo(n: u64) -> u64 {
    if n <= 1 {
        return n;            // caso base — âncora da recursão
    }
    fibonacci_recursivo(n - 1) + fibonacci_recursivo(n - 2)
}
```

Exercício 8 — Ordenação Bolha (Bubble Sort)

```python
def ordenacao_bolha(lista):
    n = len(lista)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
    return lista
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Empréstimo mutável (`&mut`) e `.swap()`

```
// Assinatura sugerida
pub fn ordenacao_bolha(lista: &mut [i32])
```

- Para modificar o conteúdo do slice dentro da função, passe por **referência mutável**: `&mut [i32]`.
- Em Python, trocar dois elementos é `a, b = b, a`. Em Rust, use o método `.swap(i, j)` diretamente no slice — é idiomático e seguro:

```
lista.swap(j, j + 1);
```

- O compilador Rust garante que não há dois empréstimos mutáveis simultâneos do mesmo dado. `.swap()` resolve isso internamente de forma segura.
- Como a função modifica o slice no local, não é necessário retornar nada (`-> ()`).

```
pub fn ordenacao_bolha(lista: &mut [i32]) {
    let n = lista.len();
    for i in 0..n {
        for j in 0..(n - i - 1) {
            if lista[j] > lista[j + 1] {
                lista.swap(j, j + 1);
            }
        }
    }
}
```

Exercício 9 — Produto de Matrizes

```
def produto_de_matrizes(A, B, n):
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Vetores bidimensionais e posse de retorno

```rust
// Assinatura sugerida
pub fn produto_de_matrizes(a: &[Vec<i64>], b: &[Vec<i64>]) -> Vec<Vec<i64>>
```

- Em Rust, uma matriz é representada como `Vec<Vec<T>>` — um vetor de vetores.
- Para criar a matriz `C` inicializada com zeros use a macro `vec!`:

```rust
let mut c = vec![vec![0i64; n]; n];
// vec![valor_inicial; tamanho]
// o vetor externo tem n linhas, cada linha tem n colunas
```

- Use `i64` para evitar overflow em multiplicações.
- As entradas `a` e `b` são passadas por referência (`&[Vec<i64>]`), pois a função apenas lê. O retorno (`Vec<Vec<i64>>`) é um valor com **posse** — o chamador receberá a posse do resultado.
- Acesso a elemento: `a[i][k]` funciona da mesma forma que em Python.

Exercício 10 — Merge Sort

```python
def merge_sort(lista):
    if len(lista) <= 1:
        return lista

    meio = len(lista) // 2
    esquerda = merge_sort(lista[:meio])
    direita  = merge_sort(lista[meio:])

    resultado = []
    i = j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    resultado += esquerda[i:]
    resultado += direita[j:]
    return resultado
```

Complexidade esperada: `O( __ )`

▶ 💡 Dicas de Rust

Posse, slices e alocação de vetores

```rust
// Assinatura sugerida
pub fn merge_sort(lista: Vec<i32>) -> Vec<i32>
```

- **Por que `Vec<i32>` e não `&[i32]`?** Merge Sort cria novos vetores a cada chamada recursiva. A função recebe **posse** da lista, divide-a em duas metades e retorna um novo vetor ordenado com **posse**.
- Fatiar e copiar para um novo `Vec`: `lista[..meio].to_vec()` e `lista[meio..].to_vec()`.
- Para a função de fusão (`merge`), use `Vec::with_capacity(n)` para alocar o espaço antecipadamente e evitar realocações.
- Para adicionar os elementos restantes ao final: `resultado.extend_from_slice(&esquerda[i..]);`
- Sugestão: extraia a lógica de fusão para uma função auxiliar privada (`fn merge(...)`).

```rust
pub fn merge_sort(lista: Vec<i32>) -> Vec<i32> {
    if lista.len() <= 1 {
        return lista;       // caso base — devolve a posse
    }
    let meio = lista.len() / 2;
    let esquerda = merge_sort(lista[..meio].to_vec());  // nova alocação
    let direita  = merge_sort(lista[meio..].to_vec());  // nova alocação
    merge(esquerda, direita)
}
```

Estrutura de Entrega

Organize o seu repositório Git da seguinte forma:

```
atividade-rust/
├── README.md              ← análise de complexidade de cada algoritmo
├── ex01_verificar_primeiro.rs
├── ex02_somar_lista.rs
├── ex03_busca_binaria.rs
├── ex04_pares_com_soma.rs
├── ex05_imprimir_pares_e_pares.rs
├── ex06_potencias_de_dois.rs
├── ex07_fibonacci_recursivo.rs
├── ex08_ordenacao_bolha.rs
├── ex09_produto_de_matrizes.rs
├── ex10_merge_sort.rs
└── prints_execucao/
    └── (screenshots ou arquivos .txt com a saída do terminal)
```

> **Dica:** use um único projeto Cargo com módulos (`mod`) para facilitar a compilação de todos os arquivos juntos.

O que deve constar no `README.md` da entrega

Para cada algoritmo, inclua:

```
## Exercício N — Nome do Algoritmo

**Complexidade:** O( __ )

**Lógica do algoritmo:**
(descreva em 2–3 linhas o que o algoritmo faz)

**Justificativa da complexidade:**
(explique quantos loops existem, como n afeta o tempo, etc.)
```

Exemplo de preenchimento

```
## Exercício 3 — Busca Binária

**Complexidade:** O(log n)

**Lógica do algoritmo:**
Divide o intervalo de busca ao meio a cada iteração, descartando
metade dos elementos restantes até encontrar o alvo ou esgotar
a lista.

**Justificativa da complexidade:**
A cada passo o espaço de busca é cortado pela metade: n → n/2 →
n/4 → ... → 1. São necessários log₂(n) passos para ir de n até 1.
Para n = 1.000.000, isso representa apenas ~20 iterações.
```

Evidências de Execução

Inclua ao menos **uma** das opções:

- **Screenshot** da saída no terminal após `cargo run`
- **Arquivo** `.txt` com a saída copiada do terminal
- **Testes automatizados** usando `#[test]` e a saída de `cargo test`

Exemplo de saída esperada para `cargo test`:

```
```

```
running 10 tests
test ex01::tests::lista_vazia ... ok
test ex03::tests::encontra_elemento ... ok
...
test result: ok. 10 passed; 0 failed
```

Critérios de Avaliação

Critério	Descrição
Compreensão do algoritmo	Demonstra entendimento do funcionamento
Análise de complexidade	Classificação Big-O correta com justificativa
Implementação em Rust	Código funcional, organizado e que compila
Uso correto de tipos Rust	Uso adequado de `&`, `&mut`, `Option`, `Vec`, etc.
Evidências de execução	Demonstração de testes ou saída no terminal
Clareza da explicação	Justificativa coerente e bem escrita no README

Por que Rust?

Rust pode parecer mais verboso que Python no início — e isso é intencional. A linguagem:

- **Privilegia segurança de memória** sem coletor de lixo (garbage collector)
- **Exige tipagem explícita** e tratamento de casos nulos com `Option<T>`
- **Torna explícita a posse e o empréstimo de dados** (`&`, `&mut`, `Vec`)
- **Detecta erros em tempo de compilação** que em outras linguagens só aparecem em produção

Essa atividade serve como primeiro contato com algoritmos em Rust — habilidade bastante valorizada em áreas como sistemas operacionais, infraestrutura de nuvem, blockchain, ferramentas de linha de comando e desenvolvimento de alto desempenho.

Referências e Recursos

- The Rust Programming Language (livro oficial) — capítulos 3, 4 e 8 são os mais relevantes para esta atividade
- Rust by Example — exemplos práticos lado a lado
- Rustlings — exercícios interativos para iniciantes
- Material da Aula 03 — Análise de Complexidade (disponível no ULife)